

Portfolio Project

David Wickline

Summary

In this project, I train a knn classification model on public perception data from GPT 3 to see if a public figure's gender sways the public's opinion of them. Try as I might, I couldn't come to a satisfying enough conclusion to let my ggplot expertise come into play.

Motivation and Context

I found a data set containing about 250 public figures and the public's opinions on them. This is always an interesting type of data, because the observations are usually recognizable, and it's neat to look at public perceptions of what the world is like.

Packages Used In This Analysis

```
library(tidyverse)
```

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v dplyr      1.2.0      v readr      2.2.0
v forcats    1.0.1      v stringr    1.6.0
v ggplot2    4.0.2      v tibble     3.3.1
v lubridate  1.9.5      v tidyr      1.3.2
v purrr      1.2.1
-- Conflicts ----- tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()     masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

```
library(tidymodels)
```

```
-- Attaching packages ----- tidymodels 1.4.1 --
v broom      1.0.12    v rsample      1.3.2
v dials      1.4.2     v tailor      0.1.0
v infer      1.1.0     v tune        2.0.1
v modeldata  1.5.1     v workflows   1.3.0
v parsnip    1.4.1     v workflowsets 1.1.1
v recipes    1.3.1     v yardstick    1.3.2
-- Conflicts ----- tidymodels_conflicts() --
x scales::discard() masks purrr::discard()
x dplyr::filter()   masks stats::filter()
x recipes::fixed()  masks stringr::fixed()
x dplyr::lag()       masks stats::lag()
x yardstick::spec() masks readr::spec()
x recipes::step()    masks stats::step()
```

```
library(kknn)
```

Warning: package 'kknn' was built under R version 4.5.3

```
library(forcats)
library(discrim)
```

Warning: package 'discrim' was built under R version 4.5.3

Attaching package: 'discrim'

The following object is masked from 'package:dials':

smoothness

```
library(klaR)
```

Warning: package 'klaR' was built under R version 4.5.3

Loading required package: MASS

Warning: package 'MASS' was built under R version 4.5.3

Attaching package: 'MASS'

The following object is masked from 'package:dplyr':

select

```
library(MASS)
library(e1071)
```

Warning: package 'e1071' was built under R version 4.5.3

Attaching package: 'e1071'

The following object is masked from 'package:tune':

tune

The following object is masked from 'package:rsample':

permutations

The following object is masked from 'package:parsnip':

tune

The following object is masked from 'package:ggplot2':

element

```
Ratings <- read_csv("DATA/Average Prolific Ratings with GPT-3 predictions.csv")
```

New names:

Rows: 226 Columns: 67

-- Column specification

----- Delimiter: "," chr
(8): name, word2vec, country, century, gender, occupation, industry, do... dbl

```
(59): ...1, index, target, TIPI_1, TIPI_2, TIPI_3, TIPI_4, TIPI_5, TIPI_...
i Use `spec()` to retrieve the full column specification for this data. i
Specify the column types or set `show_col_types = FALSE` to quiet this message.
* `` -> `...1`
```

Data Description

The observations in this dataset are public figures, and the variables are publicly available info about them as well as the average scores given to them by some n people polled. The publicly available variables aren't super interesting and fairly self explanatory.

after that, for each public figure, n people were asked to rank that figure's warmth, competence, and attitude on a scale from -100 to 100, and their extraversion, agreeableness, conscientiousness, neuroticism, and openness on a scale from 1ish to 7ish.

Data Wrangling/Exploratory Data Analysis

There are also a lot of variables in the original dataset that appear to be from the author's own work on prediction and PCA. I want to rid of those and left myself the more approachable variables I know what are. I also want to remove some of the more redundant and mysterious variables, then arrange it in a way that makes more sense to me.

```
Ratings_simpler <- Ratings |> dplyr::select("name", "country", "birthyear", "gender", "occup
set.seed(1415673)
Ratings_split <- initial_split(Ratings_simpler, prop = 0.8)
Ratings_train <- training(Ratings_split)
Ratings_test <- testing(Ratings_split)
```

I start by seeing how figures in my training data fare based on gender

```
mean_data <- Ratings_train |> group_by(gender) |> summarize(sum_n = sum(n), n = n(), warmth =
```

This is interesting, I can see which gender has (on average) better scores. I also can see how many responses were catalogued about figures for each gender as well as the number in my training set. I also see that Jennifer Lopez, Margaret Thatcher, and Elvis Presley are missing their gender. This missingness is clearly random, and I can impute the true values based on my domain knowledge.

I wonder, though: how much of the public's opinion of someone is explained just by their gender?

```
filter(Ratings_train, gender == "Missing")
```

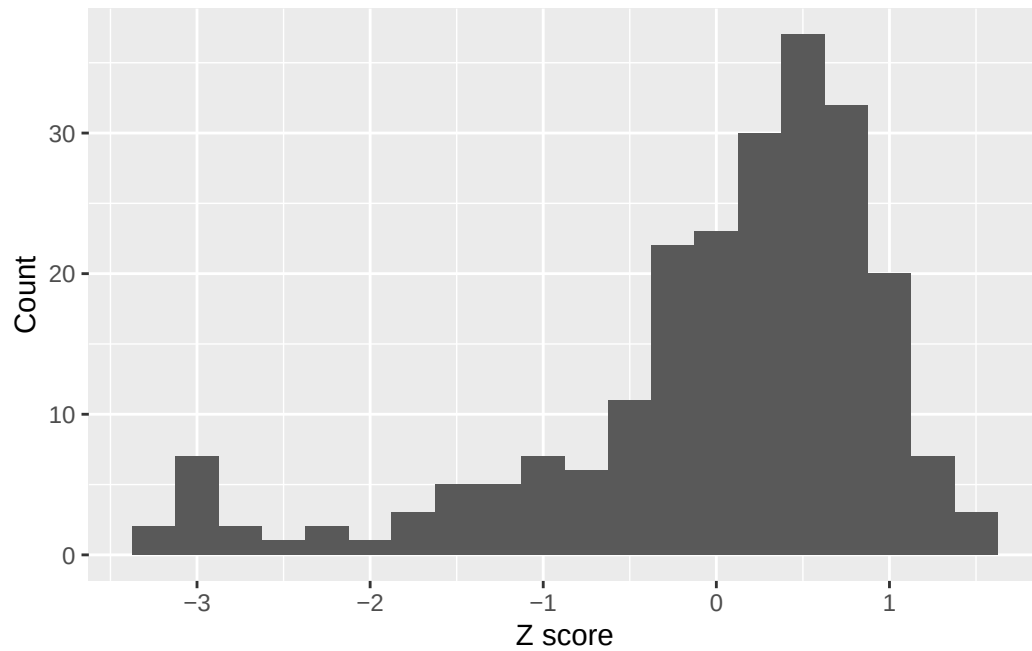
```
# A tibble: 3 x 16
  name      country birthyear gender occupation industry domain      n warmth compt
  <chr>    <fct>         <dbl> <fct>  <fct>         <fct>    <fct>  <dbl>  <dbl> <dbl>
1 Jennif~ USA          1969 Missi~ SINGER        MUSIC    ARTS    32    39.3  59.8
2 Margar~ GBR          1925 Missi~ POLITICIAN GOVERN~ INSTI~    14   -14.9  32.4
3 Elvis ~ USA          1935 Missi~ SINGER        MUSIC    ARTS    32    37.2  48.7
# i 6 more variables: attitude <dbl>, extra <dbl>, agree <dbl>, consc <dbl>,
#   neuro <dbl>, openn <dbl>
```

```
Ratings_train$gender[which(Ratings_train$gender == "Missing")] = c("Female", "Female", "Male")
```

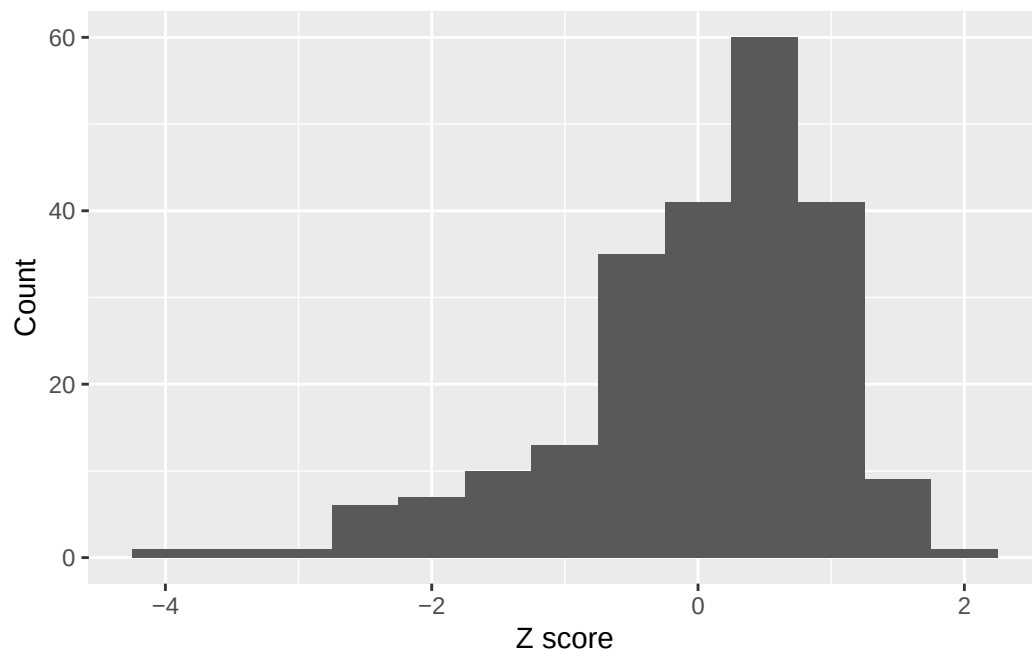
This is a question we have the tools to answer; if it is true that gender affects values of the public opinion, then a properly trained classification model would be able to predict gender accurately by looking at the public opinions. The next question is which to use?

Many of the classification models covered in class aren't designed to handle data that isn't normal, so if we can show this assumption is violated, our choices are narrowed down significantly

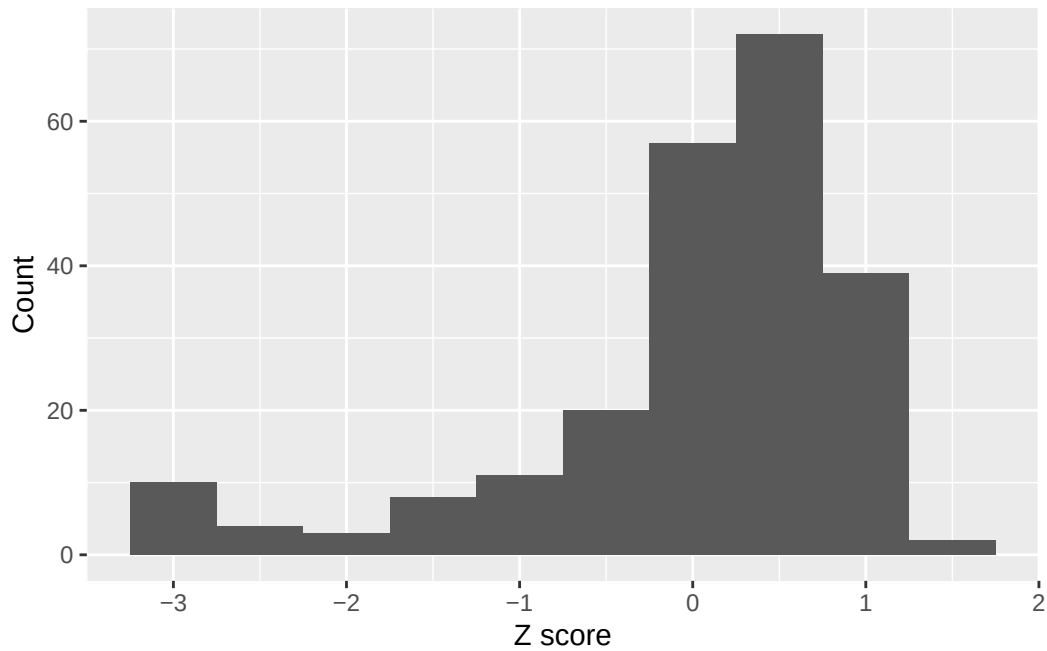
```
ggplot(data = Ratings_simpler) +
  geom_histogram(mapping = aes(x = (warmth - mean(warmth))/sd(warmth)), binwidth = 1/4, center = 0) +
  labs(y = "Count", x = "Z score")
```



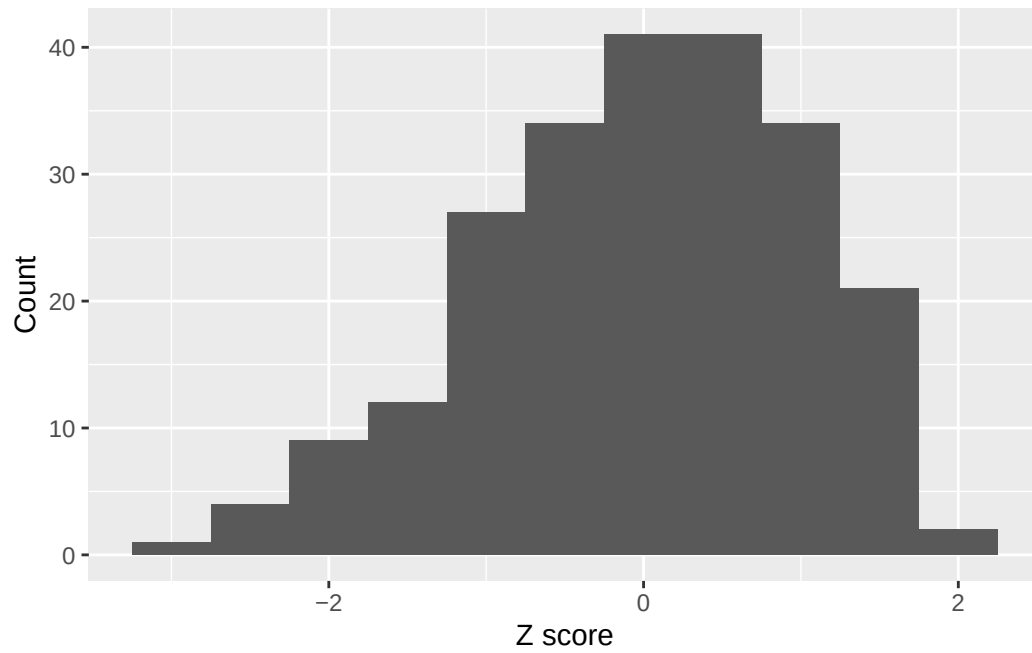
```
ggplot(data = Ratings_simpler) +
  geom_histogram(mapping = aes(x = (compt - mean(compt))/sd(compt)), binwidth = 1/2, center = 0) +
  labs(y = "Count", x = "Z score")
```



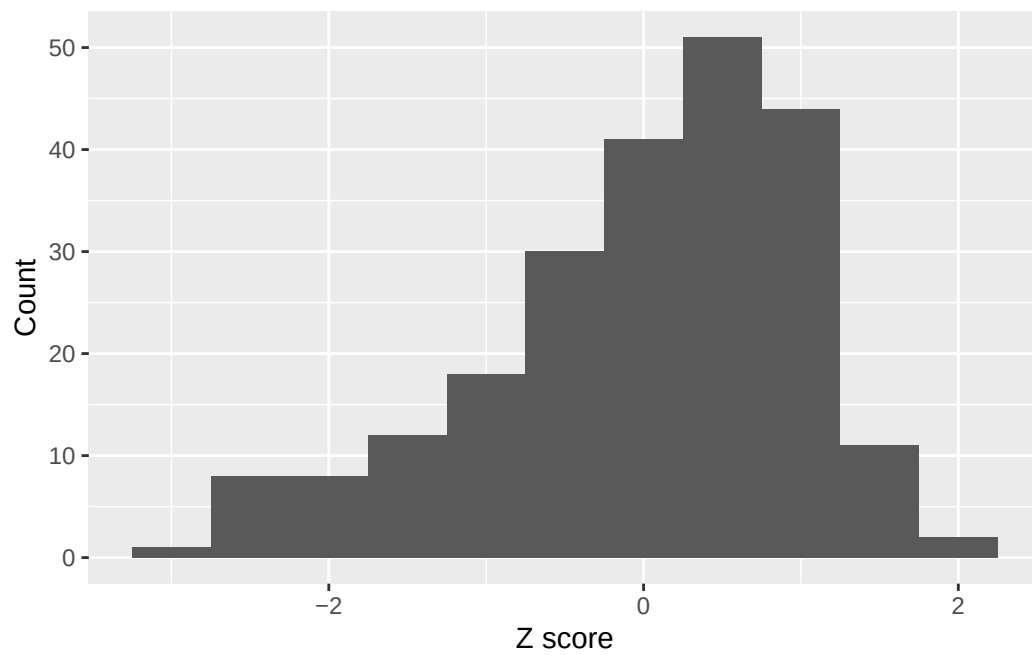
```
ggplot(data = Ratings_simpler) +
  geom_histogram(mapping = aes(x = (attitude - mean(attitude))/sd(attitude)), binwidth = 1/2,
  labs(y = "Count", x = "Z score")
```



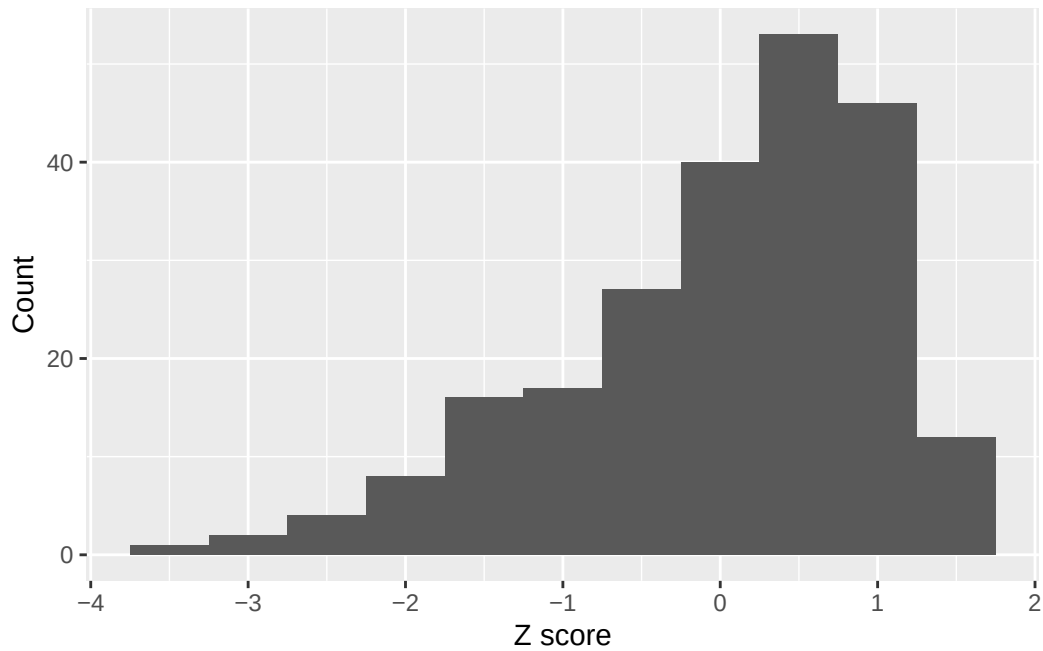
```
ggplot(data = Ratings_simpler) +
  geom_histogram(mapping = aes(x = (extra - mean(extra))/sd(extra)), binwidth = 1/2, center =
  labs(y = "Count", x = "Z score")
```



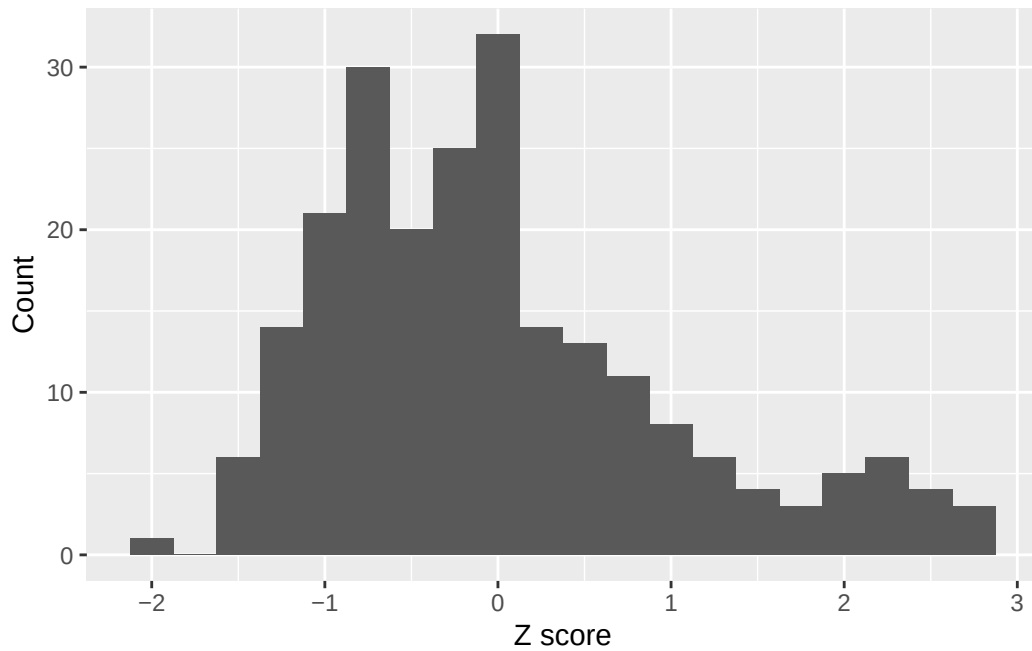
```
ggplot(data = Ratings_simpler) +  
  geom_histogram(mapping = aes(x = (agree - mean(agree))/sd(agree)), binwidth = 1/2, center = 0)  
  labs(y = "Count", x = "Z score")
```



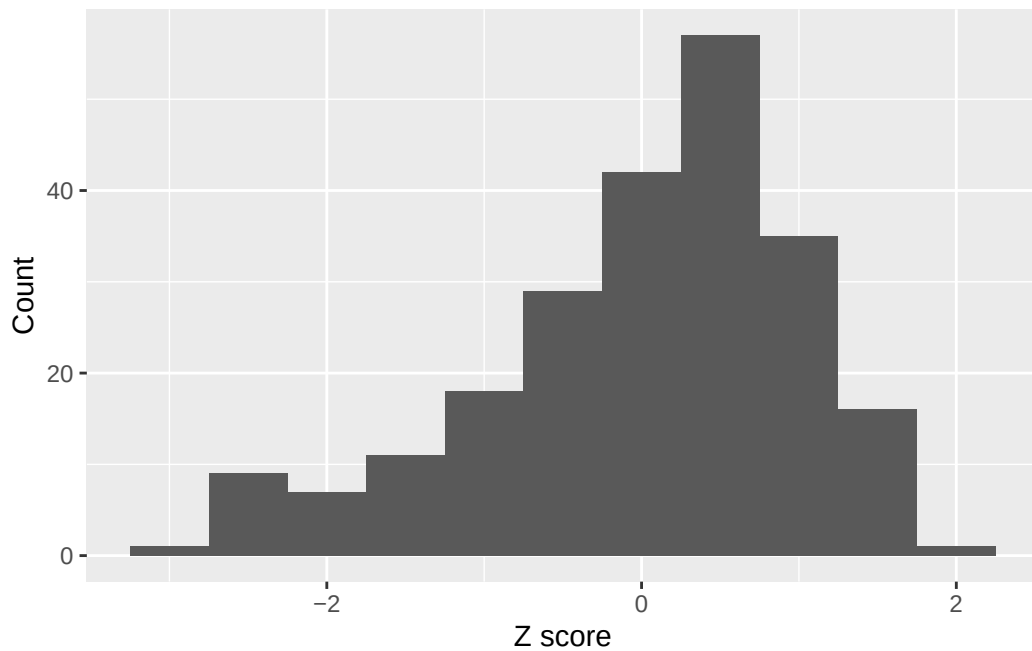

```
ggplot(data = Ratings_simpler) +
  geom_histogram(mapping = aes(x = (consc - mean(consc))/sd(consc)), binwidth = 1/2, center = 0)
labs(y = "Count", x = "Z score")
```



```
ggplot(data = Ratings_simpler) +
  geom_histogram(mapping = aes(x = (neuro - mean(neuro))/sd(neuro)), binwidth = 1/4, center = 0)
labs(y = "Count", x = "Z score")
```



```
ggplot(data = Ratings_simpler) +
  geom_histogram(mapping = aes(x = (openn - mean(openn))/sd(openn)), binwidth = 1/2, center = 0) +
  labs(y = "Count", x = "Z score")
```



Almost all of our variables are skewed (most of them are left skewed, with the exception of neuroticism; this is within expectations, as people who would score low on most of these traits don't tend to become public figures).

This skew in our opinion variables requires us to pick a classification model that can handle variables that aren't normal.

Modeling

Naive Bayes is a good contender for this, assuming our variables are independent, but k nearest neighbors can do this without that assumption. I'll use knn.

```
Ratings_knn_model_tune <- nearest_neighbor(neighbors = tune(), dist_power = 2) |>
  set_mode("classification") |>
  set_engine("kkn")

Ratings_knn_recipe <- recipe(
  gender ~ warmth + compt + attitude + extra + agree + consc + neuro + openn,
  data = Ratings_train
) |>
  step_normalize(all_numeric_predictors())

Ratings_knn_wflow_tune <- workflow() |>
  add_model(Ratings_knn_model_tune) |>
  add_recipe(Ratings_knn_recipe)
```

K nearest Neighbors predicts the value of the response variable to be the average values of the observations from the training set that are closest to it. How many neighbors are used and how distance is defined can vary, but what results is a versatile non-parametric approach to modelling.

```
set.seed(47628593)
Ratings_vfold <- vfold_cv(
  v = 7,
  data = Ratings_train
)

param.grid <- data.frame(
  neighbors = seq(from = 5, to = 145, by = 10)
)

Ratings_knn_tune <- Ratings_knn_wflow_tune |>
```

```
tune_grid(  
  resamples = Ratings_vfold,  
  grid = param.grid,  
  metrics = metric_set(mn_log_loss, roc_auc)  
)
```

```
> A | warning: x No observations were detected in `truth` for level: Missing.  
  i Computation will proceed by ignoring those levels.
```

```
There were issues with some computations    A: x15
```

```
There were issues with some computations    A: x30
```

```
There were issues with some computations    A: x45
```

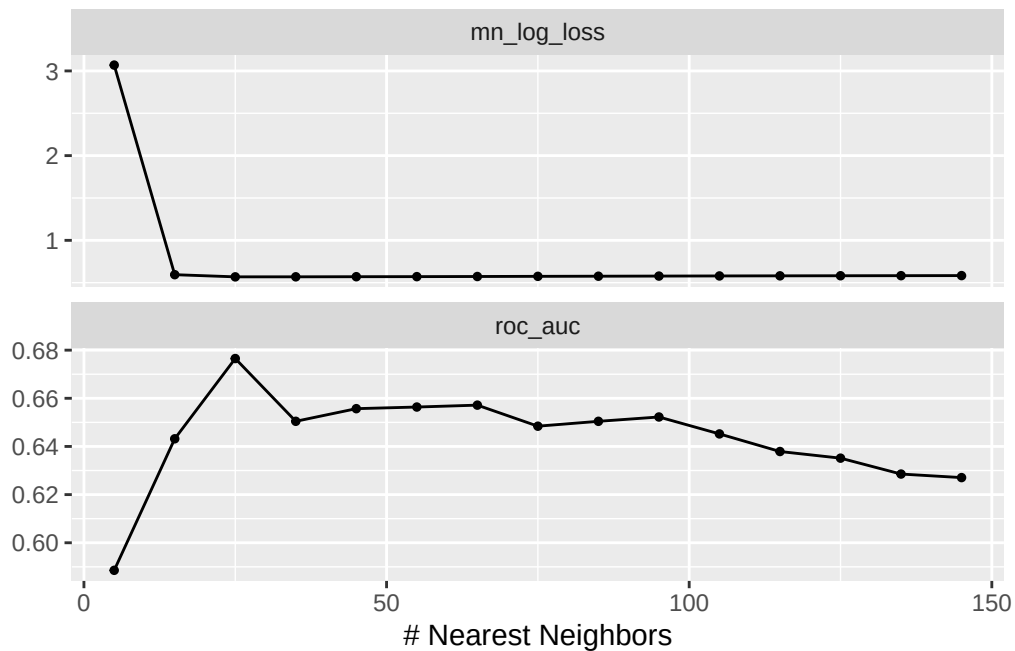
```
There were issues with some computations    A: x75
```

```
There were issues with some computations    A: x90
```

```
There were issues with some computations    A: x105
```

```
There were issues with some computations    A: x105
```

```
autoplot(Ratings_knn_tune)
```



Both of our metrics seem to cap out at 25 neighbors, so I want to fit my model with about that many neighbors

```
Ratings_knn_model_final <- nearest_neighbor(
  neighbors = 25,
  dist_power = 2
) |>
  set_mode("classification")
```

```
Ratings_knn_wflow_final <- workflow() |>
  add_model(Ratings_knn_model_final) |>
  add_recipe(Ratings_knn_recipe)
```

```
Ratings_knn_fit <- Ratings_knn_wflow_final |>
  fit(data = Ratings_train)
Ratings_knn_fit
```

```
== Workflow [trained] =====
Preprocessor: Recipe
Model: nearest_neighbor()

-- Preprocessor -----
```

1 Recipe Step

```
* step_normalize()
```

```
-- Model -----
```

Call:

```
kknn::train.kknn(formula = ..y ~ ., data = data, ks = min_rows(25, data, 5), distance = .
```

Type of response variable: nominal

Minimal misclassification: 0.2666667

Best kernel: optimal

Best k: 25

```
Ratings_knn_predictions <- Ratings_knn_fit |>
  augment(new_data = Ratings_test |> filter(gender != "Missing"))
```

Insights

```
Ratings_knn_predictions |>
  conf_mat(
    truth = gender,
    estimate = .pred_class
  )
```

	Truth		
Prediction	Female	Male	Missing
Female	2	2	0
Male	11	30	0
Missing	0	0	0

Limitations and Future Work

The public figures in this data set are overwhelmingly male, which confounded most of the work a classification algorithm could do. In the end I ended up with a model that had the same accuracy level as the null classifier. I could have operated under the misguided assumption of normality and performed a more normal test, but that didn't sit well with me. In the future, I would like to see similar data collected on a larger scale to contain more public figures.

I